

TP14 — Exposer et monitorer un service réseau dans Kubernetes

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Ce TP déploie le mock d'API réseau (TP4/TP13) dans un cluster Kubernetes local, avec plusieurs répliques (pods), un Service qui répartit le trafic entre elles, des sondes de santé automatiques, et une démonstration de l'auto-réparation de Kubernetes (un pod supprimé est recréé tout seul).

Prérequis

- TP1 à TP13 validés, Docker Desktop fonctionnel

Étapes

Étape 1 — Activer Kubernetes dans Docker Desktop

Pas besoin d'installer minikube ou kind séparément — Docker Desktop embarque son propre cluster Kubernetes local.

1. Ouvrez Docker Desktop → **Settings** (icône engrenage) → **Kubernetes**
2. Cochez **Enable Kubernetes**
3. Si l'option est disponible, préférez le mode **kubeadm** (plus standard que le mode legacy) — c'est celui qui s'est montré le plus fiable pour l'exposition des Services en pratique
4. Cliquez **Apply & Restart** (peut prendre plusieurs minutes la première fois)

Étape 2 — Vérifier que kubectl fonctionne

```
kubectl get nodes
```

Résultat attendu : un nœud en état Ready.

Étape 3 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
```

```
git pull origin dev
git checkout -b <prenom>-<nom>-jour4-tp14
```

Étape 4 — Ajouter l'identité du pod dans le mock

```
cat > scripts/mock_api.py << 'EOF'
import os
from flask import Flask, jsonify, request

app = Flask(__name__)

SITE_NAME = os.environ.get("SITE_NAME", "site-inconnu")
API_TOKEN = os.environ.get("API_TOKEN", "changeme")
POD_NAME = os.environ.get("HOSTNAME", "inconnu")

INTERFACES = {
    "GigabitEthernet0/1": {"status": "up", "ip": "10.0.1.1", "vlan": 10},
    "GigabitEthernet0/2": {"status": "down", "ip": None, "vlan": 20},
}

def check_token():
    return request.headers.get("X-API-Token") == API_TOKEN

@app.route("/health")
def health():
    return jsonify({"site": SITE_NAME, "status": "ok", "pod": POD_NAME})

@app.route("/api/devices/<device_id>/interfaces")
def get_interfaces(device_id):
    if not check_token():
        return jsonify({"error": "Token invalide ou manquant"}), 401
    return jsonify({"site": SITE_NAME, "device_id": device_id, "interfaces": INTERFACES})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
EOF
```

HOSTNAME est automatiquement défini par Kubernetes au nom du pod — ça permet de voir, en interrogeant plusieurs fois le service, que les réponses viennent de pods différents.

Étape 5 — Reconstruire l'image avec ce changement

```
cd lab
docker compose build siege-router
cd ..
```

Étape 6 — Créer les manifestes Kubernetes

```
mkdir -p k8s

cat > k8s/siege-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: siege-router
  labels:
    app: siege-router
spec:
  replicas: 2
  selector:
    matchLabels:
      app: siege-router
  template:
    metadata:
      labels:
        app: siege-router
    spec:
      containers:
        - name: siege-router
          image: lab-siege-router:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          env:
            - name: SITE_NAME
              value: "siege"
            - name: API_TOKEN
              value: "s3cr3t-token-nexacorp"
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 2
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
      
```

```

        initialDelaySeconds: 5
        periodSeconds: 10
EOF

cat > k8s/siege-service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: siege-router-svc
spec:
  type: LoadBalancer
  selector:
    app: siege-router
  ports:
    - port: 5010
      targetPort: 5000
EOF

```

imagePullPolicy: IfNotPresent est indispensable : sans ça, Kubernetes essaierait de télécharger lab-siege-router:latest depuis un registre public inexistant, au lieu d'utiliser l'image construite localement. port: 5010 est le port exposé côté extérieur du Service (évite tout conflit avec le port 5000 déjà utilisé par le lab Docker du TP4/TP13) ; targetPort: 5000 reste le port sur lequel Flask écoute à l'intérieur du conteneur, inchangé.

Étape 7 — Déployer

```

kubectl apply -f k8s/
kubectl get pods -w

```

Résultat attendu : 2 pods passent à 1/1 Running. Ctrl+C pour sortir du mode watch.

```

kubectl get deployments
kubectl get svc siege-router-svc

```

Étape 8 — Tester l'accès au service

```

curl -s http://localhost:5010/health

```

Si EXTERNAL-IP affiche une adresse privée (type 172.18.x.x) au lieu de localhost, et que curl ne répond pas : sous Windows/WSL2, le composant interne de Docker Desktop qui relie localhost au service (vpnkit-controller) peut ne pas s'activer correctement selon la configuration. Pistes, dans l'ordre : 1. Vérifier le contexte : kubectl config current-context doit afficher docker-desktop 2. Si le cluster tourne en mode legacy, basculer Docker Desktop en mode **kubeadm** (Settings > Kubernetes) et réappliquer les manifestes — ça a résolu le problème en pratique sur cette formation 3. En solution de repli, utiliser kubectl

```
port-forward : bash      kubectl port-forward service/siege-  
router-svc 5010:5010 Puis, dans un second terminal : bash      curl -s  
http://localhost:5010/health
```

Étape 9 — Observer un pod en détail

```
kubectl get pods -o wide
```

Copiez un des noms affichés, puis :

```
kubectl describe pod <nom-du-pod>
```

Regardez la section **Events** en bas (création, sondes de santé qui passent).

```
kubectl logs <nom-du-pod>
```

Doit montrer les requêtes HTTP traitées par ce pod.

Étape 10 — Démontrer l'auto-réparation de Kubernetes

```
kubectl get pods
```

Notez un nom de pod, puis :

```
kubectl delete pod <nom-du-pod>
```

```
kubectl get pods -w
```

Résultat attendu : le pod supprimé disparaît, mais un nouveau pod (nom différent) est recréé automatiquement pour respecter replicas: 2 — contrairement à un conteneur Docker classique, qui reste arrêté tant que personne ne le relance manuellement. Ctrl+C une fois stable.

Étape 11 — Vérifier la répartition de charge entre pods

kubectl port-forward ne passe pas par le vrai mécanisme de répartition de charge du Service (il reste collé à un seul pod pendant toute la durée du tunnel). Pour observer la vraie répartition, interrogez le service par son nom DNS interne, depuis un pod temporaire :

```
kubectl run test-loadbalancing --rm -it --image=curlimages/curl --restart=Never -- sh -c "for i in 1 2 3 4 5 6; do curl -s http://siege-router-svc:5010/health; echo; done"
```

Résultat attendu : le champ "pod" alterne entre les noms des différentes répliques au fil des 6 appels.

Étape 12 — Monter en charge (scaling)

```
kubectl scale deployment siege-router --replicas=3
```

```
kubectl get pods
```

Un 3ème pod doit apparaître. Revenez à 2 répliques ensuite :

```
kubectl scale deployment siege-router --replicas=2
```

Étape 13 — Committer et pousser

```
git add scripts/mock_api.py k8s/  
git commit -m "TP14 : déploiement Kubernetes du mock router (replicas, service, probes)"  
git push -u origin <prenom>-<nom>-jour4-tp14
```

Ouvrir une pull request <prenom>-<nom>-jour4-tp14 → dev.

Comprendre le TP

Deployment vs pod : vous ne créez jamais un pod directement — vous décrivez un Deployment (combien de répliques, quelle image, quelles sondes), et Kubernetes se charge de créer, surveiller et recréer les pods nécessaires pour respecter cette description en permanence. C'est la différence fondamentale avec docker compose : Compose démarre des conteneurs une fois, Kubernetes les maintient dans l'état voulu en continu.

Service et répartition de charge : un Service Kubernetes ne pointe jamais vers un pod précis (les pods sont éphémères) — il sélectionne un ensemble de pods par leurs labels et répartit le trafic entrant entre eux. `kubectl port-forward` contourne ce mécanisme (tunnel direct vers un seul pod), d'où la nécessité de tester la répartition via le DNS interne du service plutôt que via un tunnel.

Les sondes (readinessProbe/livenessProbe) sont l'équivalent Kubernetes natif des post-checks écrits à la main en Ansible (TP11) : Kubernetes appelle lui-même `/health` régulièrement, retire un pod de la rotation s'il ne répond pas correctement, et le redémarre s'il semble bloqué.

EXTERNAL-IP et vpnkit-controller : sur Docker Desktop, un composant additionnel (vpnkit-controller) est censé relier automatiquement `localhost` à l'IP interne du Service. Sur Windows avec le backend WSL2, ce composant peut se comporter de façon inconsistante selon le mode de cluster (legacy vs kubeadm) et la version de Docker Desktop — un problème d'infrastructure locale, pas une erreur de configuration Kubernetes.

Validation

- ☐ `kubectl get nodes` montre le nœud en Ready
- ☐ `kubectl get pods` montre 2 pods `siege-router` en Running/Ready
- ☐ Le service répond (directement via EXTERNAL-IP/`localhost`, ou via `port-forward` en repli)
- ☐ Supprimer un pod le fait recréer automatiquement par Kubernetes
- ☐ Le test de répartition de charge (`pod curlimages/curl` + DNS interne) montre bien une alternance entre pods
- ☐ `kubectl scale --replicas=3` fait apparaître un 3ème pod
- ☐ Une pull request <prenom>-<nom>-jour4-tp14 → dev est ouverte